

Algorithmen und Datenstrukturen



Ziele der heutigen Lerneinheit

Sie verstehen...

- ... warum Hashtabellen Suche in erwartetem $O(1)$ ermöglichen.
- ... was eine Kollision ist und warum sie bei realen Datenmengen prinzipiell unvermeidbar ist.
- ... wie Kollisionen durch Verkettung und offene Adressierung gelöst werden.

Sie können...

- ... Hashfunktionen nach der Divisions-Rest-Methode entwerfen.
- ... eine Hashfunktion anhand der Kriterien Surjektivität, Gleichverteilung, Effizienz und Unstetigkeit bewerten.
- ... Einfüge-, Such- und Löschooperationen in Hashtabellen mit Verkettung und offener Adressierung schrittweise durchführen.

Hashtabellen

Hashtabellen

Dynamische Mengen

- Wir haben mit **Arrays**, **verketteten Listen** und **Bäumen** bereits Datenstrukturen kennengelernt, die die Wörterbuch-Operationen INSERT, SEARCH und DELETE unterstützen und darüber hinaus auch das Sortieren ermöglichen.
- Kann man auf die sortierte Ausgabe der gesamten Menge verzichten, so stellt eine Hashtabelle eine effiziente Implementierungsvariante dar.
- Insbesondere kann SEARCH im besten Fall in $O(1)$ ausgeführt werden.

Hashtabellen

Prinzipielles Verfahren

- Die Speicherung der Objekte vom Typ T erfolgt in einem Array der Länge m . Dieses Array wird auch **Hashtabelle** genannt.
- Die Festlegung, an welcher Stelle in der Hashtabelle ein Objekt abgelegt wird, ist durch eine **Hashfunktion**

$$h: T \rightarrow \{0, \dots, m - 1\}$$

festgelegt.

Übung

Hashtabellen

Kollision

- Ist die Position, die von der **Hashfunktion** für ein neues Element berechnet wird, bereits belegt, so nennt man dies **Kollision**.
- Kollisionen sind grundsätzlich nicht vermeidbar, denn:
Die Mächtigkeit von T (Anzahl der möglichen zu speichernden Objekte) ist i.d.R. größer als m . Daher gilt:

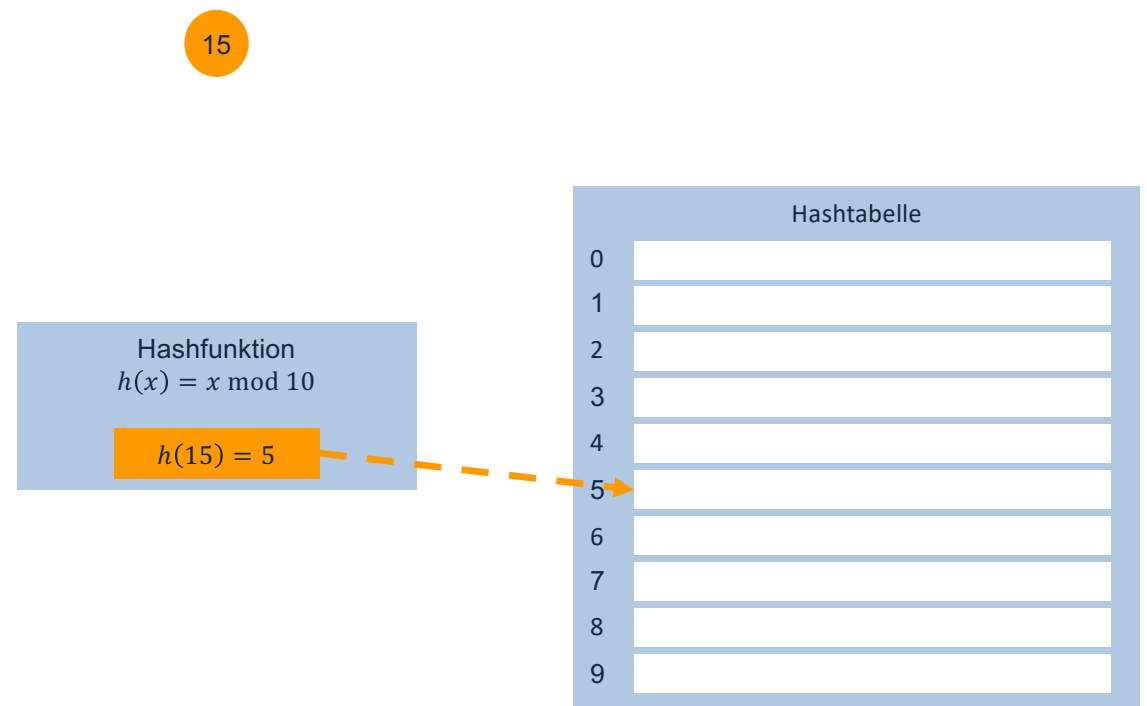
$$\exists o_1, o_2 \in T: o_1 \neq o_2 \wedge h(o_1) = h(o_2)$$

Hashtabellen

Beispiel

Gespeichert werden sollen positive ganze Zahlen in einer Hashtabelle der Größe $m=10$. Die Hashfunktion ist definiert als

$$h(x) = x \bmod 10$$

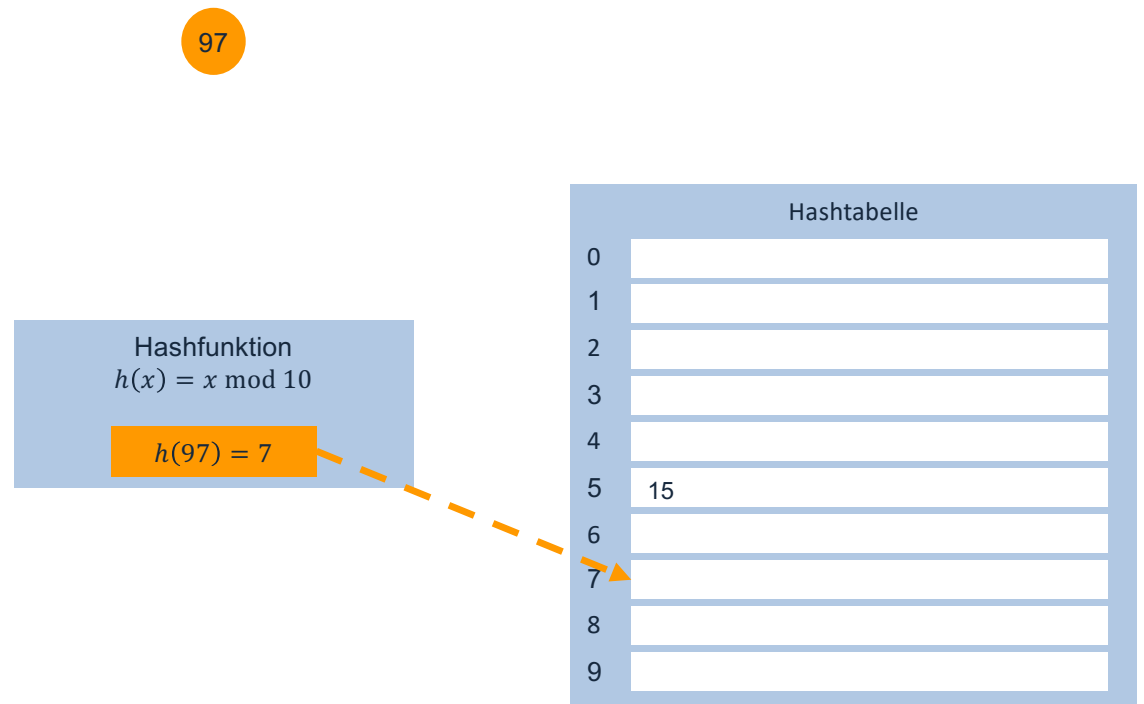


Hashtabellen

Beispiel

Gespeichert werden sollen positive ganze Zahlen in einer Hashtabelle der Größe $m=10$. Die Hashfunktion ist definiert als

$$h(x) = x \bmod 10$$



Hashtabellen

Beispiel

Gespeichert werden sollen positive ganze Zahlen in einer Hashtabelle der Größe $m=10$. Die Hashfunktion ist definiert als

$$h(x) = x \bmod 10$$

37

Hashfunktion
 $h(x) = x \bmod 10$

$$h(37) = 7$$

Hashtabelle	
0	
1	
2	
3	
4	
5	15
6	
7	97
8	
9	

Kollision

Wahl der Hashfunktion

Hashtabellen

Wahl der Hashfunktion

- Die Hashfunktion h sollte alle Plätze der Hashtabelle erreichen. (Surjektivität)
- Alle Plätze der Hashtabelle sollten mit der gleichen Wahrscheinlichkeit erreicht werden. (Gleichverteilung)
- Die Hashfunktion sollte effizient berechenbar sein. (ideal: $O(1)$)
- Ähnliche Objekte sollten nicht zu ähnlichen Hashwerten führen. (Unstetigkeit)

Hashtabellen

Wahl der Hashfunktion – Divisions-Rest-Methode

- Sind die zu speichernden Objekte positive ganze Zahlen, bietet sich die Divisions-Rest-Methode an:

$$h(x) = x \bmod m$$

- Entscheidend ist eine gute Wahl von m :

Wählt man für m eine gerade Zahl, dann ist $h(x)$ gerade, wenn x gerade ist, bzw. ungerade, wenn x ungerade ist → schlecht

- Bewährt hat sich die Wahl einer Primzahl für m

Hashtabellen

Wahl der Hashfunktion – Divisions-Rest-Methode

Zeichenketten s der Länge n können durch Aufsummieren ihrer ASCII bzw. UNICODE-Werte zu ganzen Zahlen werden:

$$h(s) = \left(\sum_{i=0}^{n-1} \text{ascii}(s[i]) \right) \bmod m$$

Probleme

- Anagramme liefern den gleichen Hashwert
$$h(\text{„logarithmus“}) = h(\text{„algorithmus“})$$
- Bei kurzen Zeichenketten werden nur relativ kleine Hashwerte erreicht

Hashtabellen

Wahl der Hashfunktion – Divisions-Rest-Methode

Verbesserung für Zeichenketten

$$h(s) = \left(\sum_{i=0}^{n-1} \text{ascii}(s[i]) \cdot 256^{n-1-i} \right) \bmod m$$

Beispiel:

$$h(\text{„test“}) = (\text{ascii}('t') \cdot 256^3 + \text{ascii}('e') \cdot 256^2 + \text{ascii}('s') \cdot 256^1 + \text{ascii}('t') \cdot 256^0) \bmod m$$

Effiziente Berechnung nach dem Horner-Schema:

$$h(\text{„test“}) = (((\text{ascii}('t') \cdot 256 + \text{ascii}('e')) \cdot 256 + \text{ascii}('s')) \cdot 256 + \text{ascii}('t')) \cdot 256) \bmod m$$

Hashtabellen

Wahl der Hashfunktion – Multiplikative Methode

Sind die zu speichernden Objekte wieder positive ganze Zahlen, so wird das Objekt mit einer irrationalen Zahl A multipliziert. Anschließend wird der gebrochene Anteil des Ergebnisses mit m multipliziert.

$$h(x) = \lfloor (x \cdot A - \lfloor x \cdot A \rfloor) \cdot m \rfloor$$

Für viele typische Eingabedaten hat sich der „goldene Schnitt“ für A bewährt:

$$A = \frac{\sqrt{5} - 1}{2} \approx 0,6180339887 \dots$$

Umgang mit Kollisionen

Hashtabellen

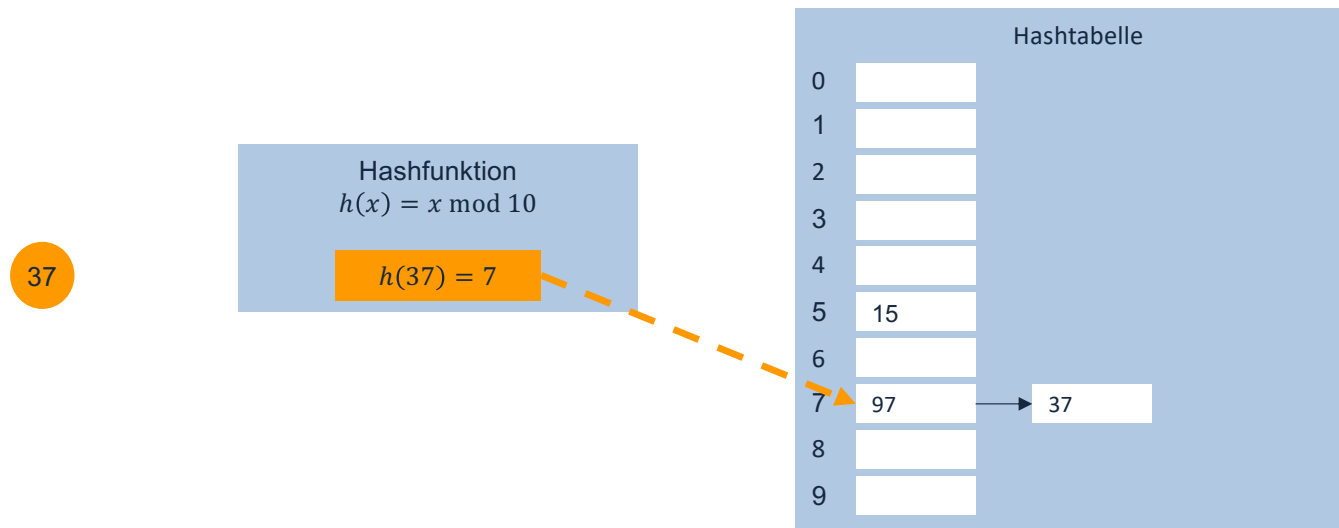
Umgang mit Kollisionen

- Kollisionsauflösung mit Verkettung
- Kollisionsauflösung durch offene Adressierung



Hashtabellen

Kollisionsauflösung mit Verkettung



- Die Hashtabelle enthält nicht die Objekte, sondern verkettete Listen der Objekte
- Die Listen sind unsortiert

Hashtabellen

Kollisionsauflösung mit Verkettung

Einfügen in eine Hashtabelle mit Verkettung ist in $O(1)$ möglich, da das Element an den Anfang der Liste gehängt werden kann.

Bei der **Suche** nach einem Objekt ist mit einer mittleren Länge der Listen von n/m (auch Belegungsfaktor α genannt) zu rechnen. Die durchschnittliche Zeitkomplexität zum Auffinden eines Elements in der Hashtabelle ist hier dann $O(1 + n/m)$

Das **Löschen** ist in konstanter Zeit möglich, wenn bereits das zu löschende Listenelement bekannt ist und die Liste doppelt verkettet ist.

Hashtabellen

Kollisionsauflösung mit offener Adressierung

Die Hashfunktion liefert nicht eine Adresse, sondern eine Folge von Adressen, an denen das Objekt abgelegt werden kann.

$$h : T \times \{0, \dots, m-1\} \\ \rightarrow \{0, \dots, m-1\}$$

37

Hashfunktion
 $h(x, i) = (x + i) \bmod 10$

$$h(37, 0) = 7$$

Hashtabelle	
0	
1	
2	
3	
4	
5	15
6	
7	97
8	
9	

Hashtabellen

Kollisionsauflösung mit offener Adressierung

Die Hashfunktion liefert nicht eine Adresse, sondern eine Folge von Adressen, an denen das Objekt abgelegt werden kann.

$$h : T \times \{0, \dots, m - 1\} \\ \rightarrow \{0, \dots, m - 1\}$$

Hashfunktion
 $h(x, i) = (x + i) \bmod 10$

$h(37, 1) = 8$

Hashtabelle	
0	
1	
2	
3	
4	
5	15
6	
7	97
8	37
9	

Hashtabellen

Kollisionsauflösung mit offener Adressierung

Bei Verwendung von

$$h(x, i) = (h'(x) + c_1 i + c_2 i^2) \bmod m$$

spricht man von ***quadratischem Sondieren***.

$h'(x)$ ist eine übliche Hashfunktion, c_1 und c_2 zwei Konstanten ungleich 0, die so gewählt werden, dass die Sondierungssequenz alle Slots erreicht.

- Quadratisches Sondieren vermeidet die (primäre) Clusterbildung besser als lineares Sondieren.
- Allerdings: Für zwei Schlüssel x_1 und x_2 mit $h'(x_1) = h'(x_2)$ ergibt sich auch hier die gleiche Sondierungssequenz (sekundäre Clusterbildung).

Hashtabellen

Kollisionsauflösung mit offener Adressierung

Bei der Analyse des Laufzeitverhaltens wird von einer optimalen Hashfunktion ausgegangen. In diesem Fall ist mit höchstens

$$\frac{1}{1-\alpha} \text{ Sondierungen}$$

bei einem Zugriff zu rechnen.

Suchen, **Einfügen** und **Löschen** ist damit in $O(1)$ möglich, wenn α als konstant und hinreichend klein angesehen werden kann.

Beim Löschen eines Objekts kann der Slot nicht einfach auf **nil** gesetzt werden, weil dadurch ggf. eine spätere Suche unterbrochen wird. Stattdessen sollte je Slot ein Zustand „besetzt“, „noch nie besetzt“ und „wieder frei“ geführt werden.

Vielen Dank für die Aufmerksamkeit!